

Recherche de documents « Document Retrieval »

Séance 1

I. Introduction :

Avant de pouvoir mettre en place un moteur de recherche capable de retrouver des documents pertinents, il est indispensable de disposer d'un corpus bien structuré et cohérent. La manière dont les documents sont collectés, nommés et organisés aura un impact direct sur les étapes ultérieures, telles que l'extraction de caractéristiques, la création d'index et la recherche par similarité.

Lors de cette première séance, les étudiants vont collaborer pour constituer un petit corpus textuel diversifié, explorer ses contenus et mettre en place des routines Python simples pour manipuler automatiquement les fichiers. L'objectif est de comprendre la structure du corpus, détecter ses éventuelles incohérences et le préparer pour les étapes suivantes du projet, le tout **sans recourir à des bibliothèques spécialisées**. Cette approche permet de se familiariser avec les aspects pratiques de la gestion de documents, tout en posant les bases d'un moteur de recherche fonctionnel.

Plan de la première étape de l'atelier :

1. Constitution et structuration du corpus :

a) Collecte de documents

La première étape du projet consiste à constituer le corpus textuel sur lequel reposera le mini-moteur de recherche documentaire. Deux configurations complémentaires sont proposées afin d'explorer progressivement différents niveaux de complexité dans la gestion et la représentation des documents.

1) Cas 1 – Corpus de phrases (niveau d'introduction)

Dans ce premier scénario, un seul fichier texte représente le corpus complet. Chaque **phrase** de ce fichier est considérée comme un document indépendant.

Deux corpus sont fournis :

- un fichier en français contenant les discours de **Jacques Chirac** lors de sa réélection ;
- un fichier en anglais reprenant les discours de **Barack Obama** pour son second mandat.

Ce cas simple servira d'introduction aux notions de segmentation, de lecture et parcours de fichier, et de représentation élémentaire des documents.

Il permettra de se familiariser avec la structure d'un corpus avant de passer à des données plus complexes et réalistes.

2) Cas 2 – Corpus de textes complets (niveau avancé)

Dans ce second scénario, l'objectif est de construire un corpus bilingue structuré à partir des productions des étudiants.

Chaque étudiant doit rédiger un texte en français, puis en produire la traduction en anglais.

Le sujet de rédaction est le suivant :

Vous êtes désormais étudiants en Master. Prenez un moment de recul pour analyser et évaluer votre parcours de formation antérieur. Rédigez un texte d'environ une page dans lequel vous exprimez votre ressenti global sur cette expérience.

Votre réflexion devra inclure :

- les points positifs que vous avez particulièrement appréciés (contenus, encadrement, méthodes, ambiance, etc.) ;
- les aspects qui pourraient être améliorés selon vous ;
- ainsi que les difficultés, limites ou points négatifs que vous avez pu rencontrer.

L'objectif est de développer une analyse critique personnelle, équilibrée et constructive de votre parcours avant le Master.

Dans votre promotion, plusieurs parcours se croisent : UFR, IUT, universités d'autres régions ou d'autres pays. Nous allons tirer parti de cette diversité pour créer un corpus structuré en quatre sous-corpus, chacun représentant une origine de formation différente.

Chaque texte, accompagné de sa version traduite, constitue un document individuel au sein du corpus. L'ensemble de ces productions forme ainsi un corpus bilingue structuré, organisé à la fois par étudiants et par langue. Cette organisation permettra non seulement d'étudier les **variations linguistiques et stylistiques** selon les parcours de formation, tout en ouvrant la voie à des expérimentations plus avancées telles que :

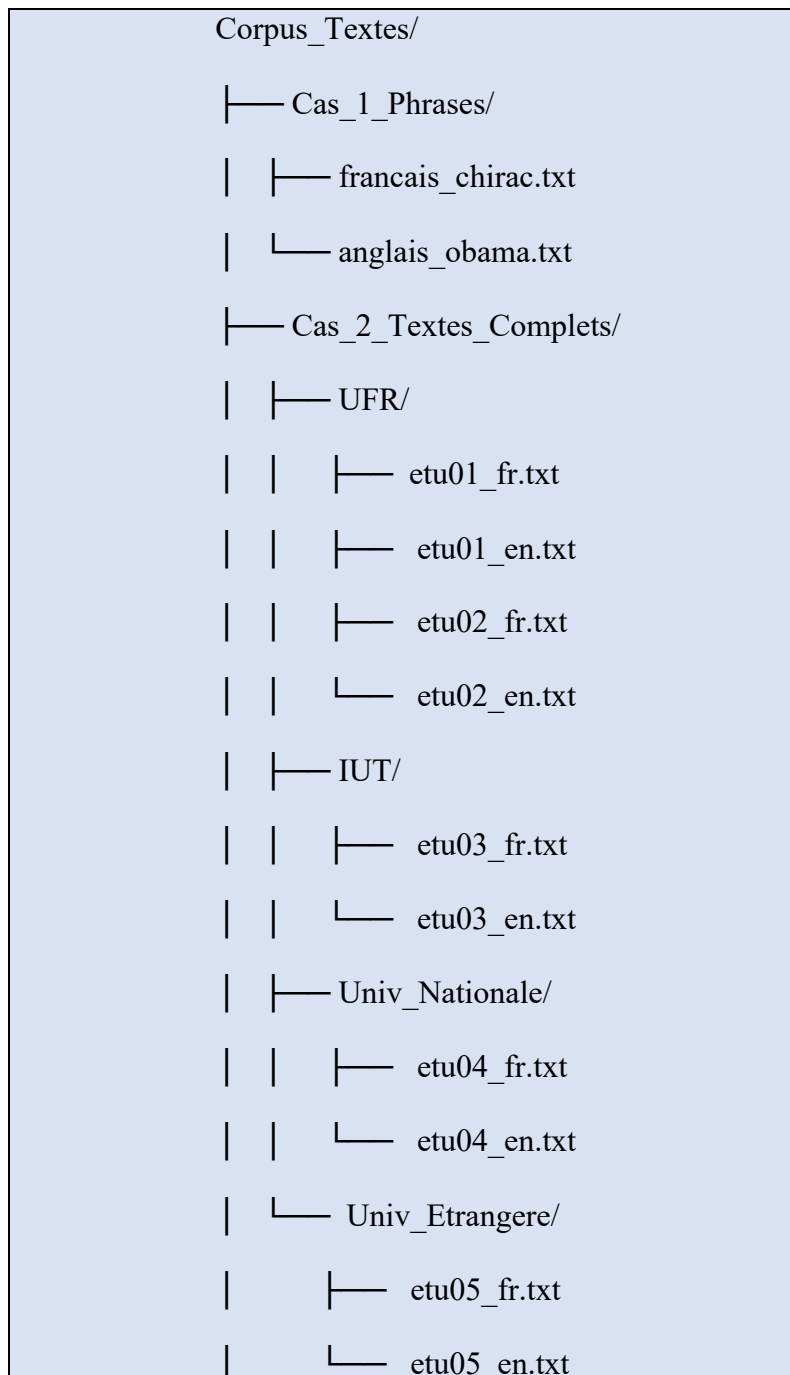
- la recherche documentaire multilingue,
- la recherche croisée entre langues,
- ou encore la comparaison sémantique inter-langues.

b) Organisation hiérarchique :

Une fois les documents collectés, il est essentiel d'adopter une organisation hiérarchique claire et cohérente de la base textuelle. Cette structuration facilite la

navigation dans le corpus, la lecture automatique des fichiers et les traitements à venir (indexation, analyse statistique, extraction de caractéristiques, etc.).

Structure générale recommandée :



Conventions de nommage :

Chaque sous-dossier correspond à un sous-corpus associé à une catégorie d'origine de formation : UFR, IUT, Univ_Nationale, Univ_Etrangere.

Les fichiers sont nommés selon la convention suivante :

etuXX_lang.txt.

où

- XX représente l'identifiant de l'étudiant sur deux chiffres (01, 02, 03, ...);
- lang indique la langue du document :
 - fr pour la version française,
 - en pour la version anglaise.

Avantages de cette organisation :

- Elle assure une **cohérence globale** dans la structuration du corpus.
- Elle permet une **lecture automatisée** et un parcours hiérarchique uniforme du corpus.
- Elle rend possible le chargement sélectif des documents par langue, par étudiant ou par groupe d'origine.
- Elle facilite l'indexation et l'exploration à venir, en distinguant clairement les langues **et les** origines de formation.

Cette organisation servira de base à la prochaine étape du projet : l'exploration du corpus, où vous écrirez des fonctions Python pour parcourir automatiquement les dossiers, vérifier la conformité des fichiers et analyser la composition de la base.

2. Exploration du corpus

Une fois la base de documents construite et organisée, il est essentiel de l'explorer pour mieux comprendre sa structure, sa composition et ses éventuelles irrégularités. Cette étape a un double objectif :

- Vérifier la cohérence du corpus (nombre de documents, noms, langues, sous-dossiers) ;
- Préparer le terrain pour les traitements textuels à venir (nettoyage, indexation, représentation vectorielle).

Vous allez progressivement développer des fonctions Python pour automatiser l'inspection et l'analyse du corpus, sans recourir à des bibliothèques externes de traitement du langage naturel.

a) Fonctions utilitaires pour naviguer dans la base :

Ces premières fonctions visent à parcourir le corpus, repérer les fichiers et afficher les informations essentielles.

- i. Écrire une fonction **choisir_document()** qui ouvre une boîte de dialogue permettant à l'utilisateur de sélectionner un fichier texte dans son ordinateur, puis retourne le chemin complet du document choisi.
- ii. Écrire une fonction **lister_document(chemin)** qui liste tous les fichiers texte contenus dans un répertoire donné.

- iii. Écrire une fonction **explorer_corpus(chemin_base)** qui parcourt récursivement l'ensemble de la base, affiche les sous-répertoires et le nombre de fichiers trouvés dans chacun.
- iv. Écrire une fonction **afficher_structure(chemin_base)** qui produit un affichage hiérarchique (indenté) de la structure du corpus pour visualiser l'arborescence complète.

b) Fonctions d'analyse statistique de base (sans lecture du contenu des fichiers)

Ces fonctions permettent de collecter des informations globales sur la composition du corpus sans analyser le contenu textuel.

- i. Écrire une fonction **compter_sous_corpus(base)** qui retourne le nombre de sous-corpus (UFR, IUT, etc.) et leurs noms.
- ii. Écrire une fonction **compter_documents(base)** qui compte le nombre total de documents dans le corpus, toutes catégories confondues.
- iii. Écrire une fonction **verifier_correspondance_langues(base)** qui vérifie que chaque étudiant possède bien un document en français et sa version en anglais. Elle affichera un message d'alerte si un fichier est manquant ou mal nommé.
- iv. Écrire une fonction **compter_par_langue(base)** qui calcule le nombre de documents en français et en anglais, à partir du nom des fichiers (_fr.txt, _en.txt, etc.).
- v. Écrire une fonction **repartition_langue_par_sous_corpus(base)** qui produit un tableau ou un graphique affichant, pour chaque sous-corpus, la proportion de documents français et anglais.
- vi. Écrire une fonction **detecter_doublons(base)** qui vérifie s'il existe des fichiers portant le même nom (ou presque identique) dans plusieurs sous-corpus.
- vii. Écrire une fonction **verifier_extensions(base)** qui vérifie que tous les fichiers respectent le format attendu (ex. .txt) et signale les anomalies éventuelles.
- viii. Écrire une fonction **compter_etudiants(base)** qui estime le nombre d'étudiants représentés dans le corpus (en supposant qu'un étudiant = un couple de fichiers FR/EN).
- ix. Écrire une fonction **statistiques_structure(base)** qui produit un petit rapport résumé :
 - nombre de sous-corpus,
 - nombre total de fichiers,
 - nombre de fichiers français / anglais,
 - nombre d'étudiants identifiés,
 - nombre d'anomalies détectées.

c) Fonctions de visualisation et tableau de bord (sans lecture du contenu des fichiers)

- i. Écrire une fonction **afficher_repartition_sous_corpus(base)** qui affiche un diagramme en barres représentant le nombre de documents par sous-corpus (UFR, IUT, etc.).
- ii. Écrire une fonction **afficher_repartition_langues(base)** qui produit un graphique circulaire (camembert) indiquant la proportion de textes français et anglais dans l'ensemble du corpus.
- iii. Écrire une fonction **afficher_repartition_langues_par_sous_corpus(base)** qui génère un diagramme groupé (par sous-corpus) où chaque barre indique le nombre de textes français et anglais.
- iv. Écrire une fonction **tableau_de_bord_corpus(base)** qui combine les visualisations précédentes (barres, camemberts, etc) dans un même graphique.